intuitive

BUSINESS REQUIREMENTS DOCUMENT

Agent Dashboard (AI Garage Agent Dashboard)

Field	Value
Document ID	BRD-AGD-001
Version	1.0
Date	21 October 2026
Author	Arya Kayastha
Status	Draft — pending stakeholder review
Scope of this document	Scopes Phase 2a and Phase 2b enhancements across the agent registry, role-based workflows, FinOps monitoring, and autonomous telemetry ingestion
Target platforms	React SPA (browser), Node.js Express API on Azure Container Apps, Databricks SQL (Unity Catalog Delta tables)
Reference	[Reference/supersedes — none for v1.0]

Top Five Features

#	Feature	The question it answers	Section
1	Contributor Workflow & Review Queue	Has a verified agent lead reviewed and authorized the latest status and progress claims before they show on the portfolio dashboard?	4.2
2	Blocker Escalation & Slip Tracking	Exactly why did an agent's completion date slip, and which blockers have sat unresolved for more than 3 days without leadership escalation?	4.2

intuitive

#	Feature	The question it answers	Section
3	Per-Agent Telemetry & Heartbeat Monitoring	Which production agents are currently running and healthy across our multi-cloud footprint, and which have stopped reporting in the last 15 minutes?	4.4
4	FinOps Cost & Reliability Analytics	How much money is each individual agent spending across LLM providers, and are any running with unacceptable error rates or unmapped model costs?	4.3
5	Role-Based Access & Identity Security	Can contributors modify or view sensitive agent records they do not own, and are all account credentials protected against brute-force attacks?	4.1

1. Purpose

Intuitive's AI Garage team is building and deploying dozens of AI agents concurrently. These workloads are maintained by disparate owners and run on decentralized environments spanning Microsoft Azure, Amazon Web Services (AWS), Google Cloud Platform (GCP), and local development workstations. Currently, the organization lacks a single, trusted system of record to monitor each agent's execution status, progress, target delivery dates, critical blockers, runtime reliability, and operational cost. Executive leadership cannot evaluate portfolio-wide health without initiating ad-hoc status requests; the Agent Manager must manually chase assignees for narrative updates; contributors lack a structured channel to report incremental progress; and delivery roadblocks or schedule slips are detected only after deadlines have passed.

Phase 1 established an initial presence on Azure Container Apps backed by Databricks SQL, but relied on a shared-password model where the Agent Manager had to enter every system update manually. This Business Requirements Document (BRD) scopes Phase 2 (Phases 2a and 2b) of the Agent Dashboard. Phase 2 transitions the platform to isolated personal accounts, self-service contributor status reporting governed by formal review and approval workflows, automated delivery slip

intuitive

and blocker escalations, and automated machine-to-dashboard telemetry for cost, error tracking, and heartbeat monitoring.

Because enterprise IT permissions are unavailable—precluding Entra ID application registrations, provisioned organizational exchange mailboxes, and corporate DNS modifications—this document defines requirements for an autonomous operational architecture. Every capability specified herein delivers governance, visibility, and FinOps accountability without introducing external IT dependencies.

2. Objectives

ID	Objective	Success measure
OBJ-01	One source of truth	100% of active agents tracked with status, progress, owner and target date. Measure: agents tracked vs. agents known.
OBJ-02	Fresh data	Every active agent has an approved update in the last 7 days (the window is a suggestion). Measure: % of agents with a recent update.
OBJ-03	Accountable updates	Assignees send updates, and only the Agent Manager's approval changes the dashboard. Measure: % of updates reviewed within 2 business days (suggested target).
OBJ-04	Early warning	Date slips and blockers are visible and escalated automatically. Measure: blockers of 3+ days escalated 100% of the time.
OBJ-05	Cost and reliability visibility	Usage, cost and errors per agent, with alerts. Measure: % of production agents reporting usage.

intuitive

3. Scope

3.1 In scope

- Agent register: create, edit and delete agents; assign owners; marketplace flag
- Dashboard with status, progress, ETA, health filters and monthly calls/cost/error rate
- Personal accounts: one-time invite links (48 h), password reset (1 h), lockout, role-based access
- Contributor updates with Agent Manager review: approve, edit then approve, request changes, not accept, bulk approve
- Daily Update Sheet, update history and progress trend chart
- Target date (ETA) history, slip badges, mandatory reason when a date moves later
- Mandatory blocker reason and unblocker; automatic escalation after 3 days of being blocked
- Audit log of every change to an agent
- FinOps: usage ingestion API, effective-dated model pricing, alerts and thresholds
- Portfolio Insights with month-over-month trend and anomaly detection
- Agent-to-dashboard connection (2b): per-agent API keys, HTTPS push, 5-minute heartbeat, offline alerts
- Unit tests, integration suites, CI and Azure deployment

3.2 Explicitly deferred

The following items are deferred from Phase 2 and prioritized as candidates for Phase 3. See Section 10 (Future Scope) for detailed technical and prerequisites analysis:

- Weekly executive summary email (Phase 3; needs 2a/2b data first)
- Microsoft Teams notifications (Phase 3; avoids new tenant permissions for now)
- A2A test tasks (Phase 3; Phase 2 is visibility only)
- Control features (start/stop agents) (Phase 3; visibility must be trusted first)
- Microsoft sign-in (SSO) (Needs an Entra app registration, which requires IT)
- Multi-role users / role switcher (One role per person is simpler; assignment already covers most needs)
- CI running integration tests (Decision not to store Databricks secrets in GitHub; run locally before each deploy)

intuitive

3.3 Permanently excluded (not a phasing decision)

The following capabilities will not be built into the Agent Dashboard platform regardless of phase, architecture evolution, or timeline:

- Connecting directly to agents' own databases or infrastructure (agents push over HTTPS only)
- Storing agents' business data or prompts
- Replacing project-management tools (tasks, sprints)
- Executives editing data

This exclusion is permanent: the Agent Dashboard serves exclusively as a metadata aggregation, visibility, and FinOps governance platform. It does not inspect workload payloads, manage underlying operational data stores, or supersede task-level issue tracking.

---

4. Detailed Requirements

4.1 Identity, Access Control & User Administration

ID	Requirement	Priority
BR-02	Sign in with personal accounts via one-time invite links (48 h) and password reset links (1 h)	M
BR-03	Role-based access (Contributor, Agent Manager, Executive, FinOps) enforced by the API	M
BR-04	Contributors see and update only their assigned agents; other agents return 404	M
BR-19	Teams page to invite people, set roles and deactivate accounts (ends sessions immediately)	M

intuitive

4.2 Agent Registry, Review Workflows & Schedule Governance

ID	Requirement	Priority
BR-01	Create, edit and delete agents with name, description, assignees, ETA, status and progress	M
BR-05	Contributor updates go to review; only approval changes status and progress	M
BR-06	Reviewer can approve, edit then approve, request changes or not accept; a comment is required for the last two and is shown to the assignee	M
BR-07	Reviewer can approve several updates at once	S
BR-08	Agent Manager's own updates apply directly and are credited to them	M
BR-09	Moving an ETA later requires a reason; ETA history and a slip badge (e.g. "Slipped 3× • +24 days") are shown	M
BR-10	Setting an agent to Blocked requires a reason and who or what can unblock it; blocked duration is shown	M
BR-11	A blocker open 3+ days (configurable) triggers one email to Agent Managers, a dashboard highlight and an audit entry, once per block	M
BR-12	Append-only audit log with before/after values, filterable by agent and person	M
BR-13	Daily Update Sheet bulk editor (one entry per agent per day)	S
BR-14	Progress history chart and full update history	S

intuitive

4.3 FinOps Cost Management & Portfolio Insights

ID	Requirement	Priority
BR-15	Usage ingestion API (single and batch) with automatic cost calculation	M
BR-16	Effective-dated model pricing; alerts for expensive calls, high error rates and unpriced models	M
BR-17	Alert thresholds, global or per agent; alert bell with open-alert count	S
BR-18	Portfolio Insights with month-over-month trend, anomaly detection and computed summary	S
BR-23	AI-written narrative via the Anthropic API when a key is set, with a computed fallback	C

4.4 Agent Telemetry & Ingestion (Phase 2b)

ID	Requirement	Priority
BR-20	Agents push a heartbeat every 5 minutes with a per-agent API key; offline after 15 minutes (production only)	M
BR-21	Offline alerts: at most two emails per outage (offline, then back online after 15 minutes stable) to assignees and Agent Manager	S
BR-22	Copy-paste integration snippets in several languages	S

---

5. Solution Approach

5.1 Process Flow

A user accesses the single-page application and authenticates using their personal credentials, obtaining a server-validated session token. When a contributor submits

progress, ETA changes, or operational status for an assigned agent, the Express API validates the payload with Zod schemas, records the submission in a pending state, and updates the UI status badge to "Pending review" without altering live dashboard progress. The Agent Manager accesses the review queue to approve, modify and approve, request changes, or reject the submission (mandatory feedback required for non-approvals). Upon approval, the engine applies the changes to the active agent record, appends an entry to the immutably tracked update and ETA history tables, writes an audit record containing before-and-after states, and emits an email notification if an escalation rule is tripped. In parallel, runtime machine agents ingest heartbeats and usage records via secure HTTPS endpoints using per-agent API keys, triggering the FinOps engine to compute costs against effective-dated pricing models, evaluate anomaly thresholds, and persist metrics to Databricks SQL.

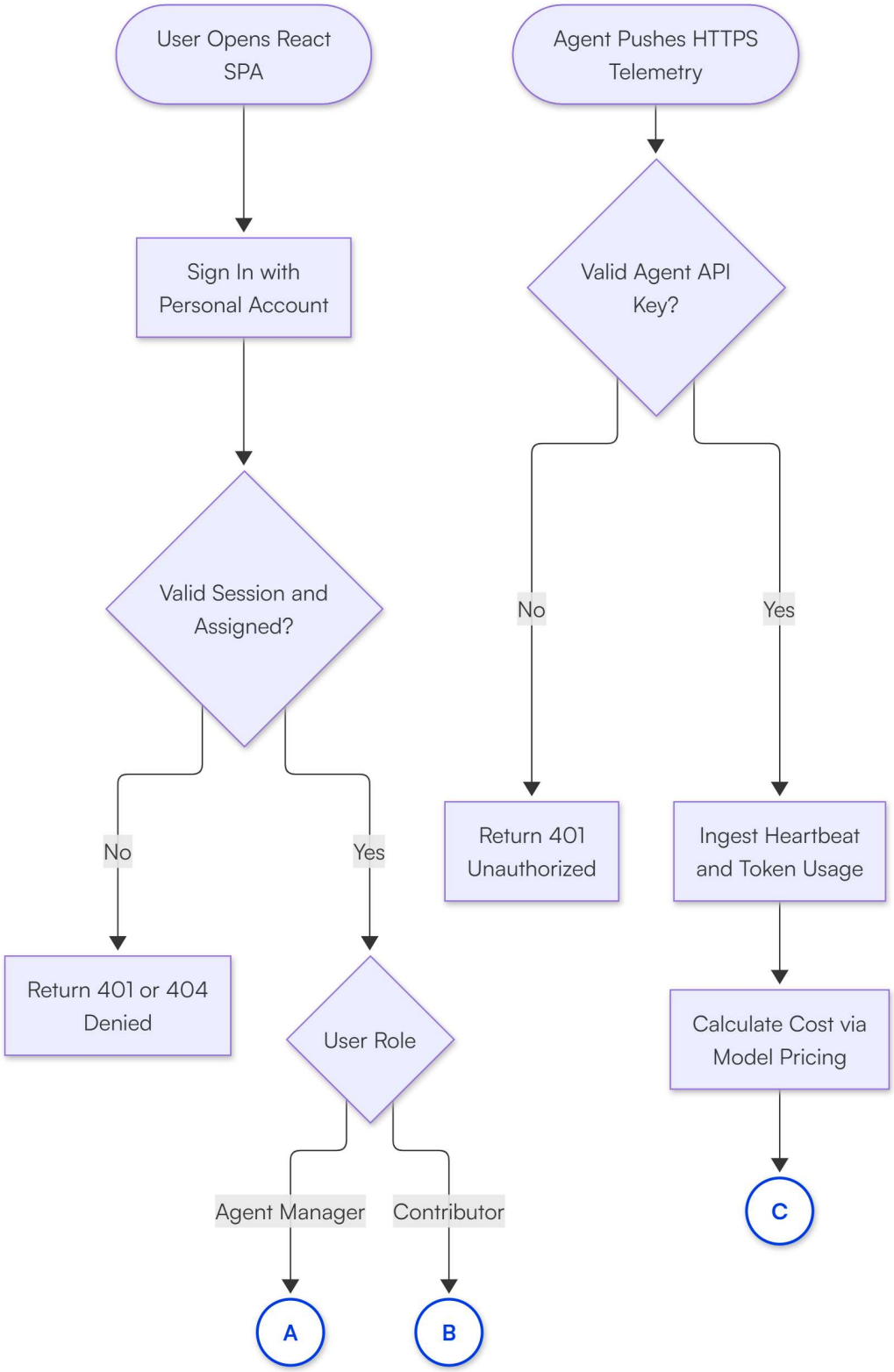


Figure 1 — Process Flow (part 1 of 4)

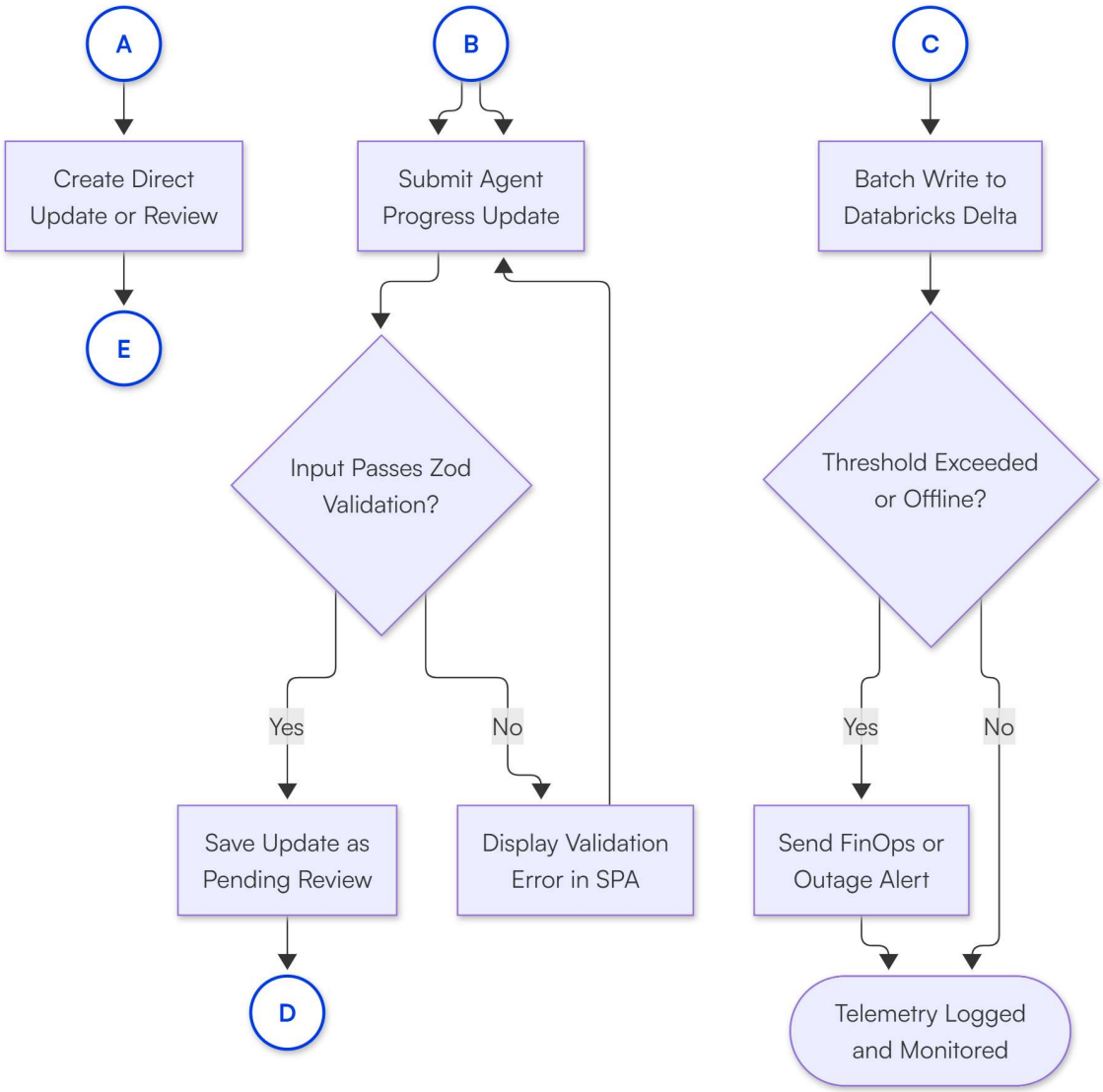


Figure 1 — Process Flow (part 2 of 4)

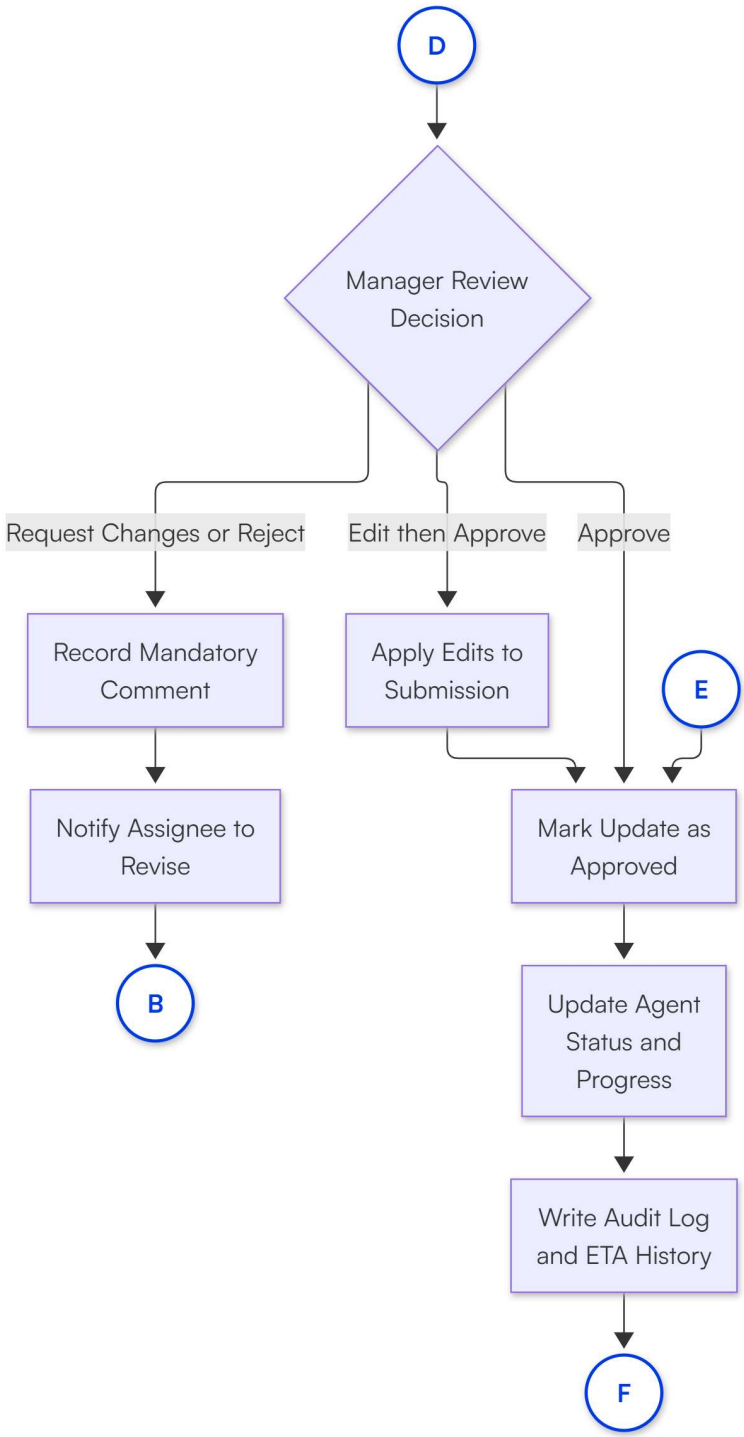


Figure 1 — Process Flow (part 3 of 4)

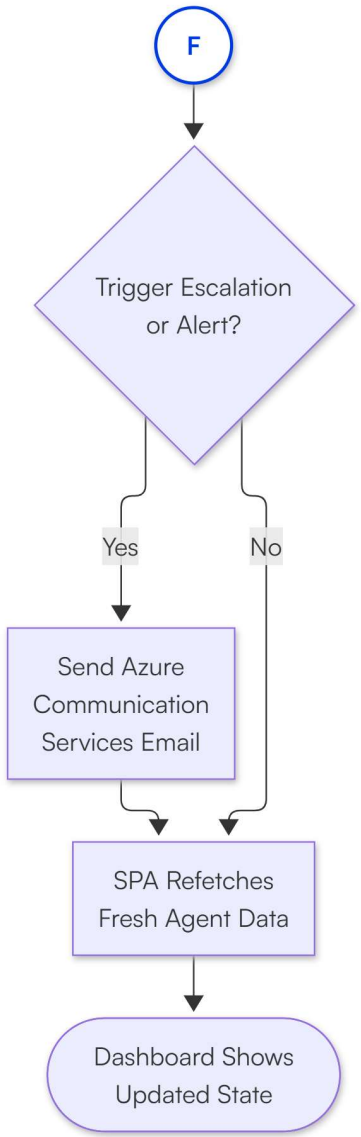


Figure 1 — Process Flow (part 4 of 4)

5.2 Architecture

The solution executes as a unified containerized deployment on Azure Container Apps, co-hosting the React 19 single-page application and the Node.js Express backend service from a single origin. The Express API acts as the core orchestration boundary, routing incoming traffic through centralized session authentication, role-based authorization guards, and rate-limiting middleware before hitting business logic services. These modular backend services—covering identity and authentication, update submissions, schedule tracking and escalation, append-only auditing, and FinOps analytics—coordinate all relational and time-series writes to Databricks SQL warehouse Delta tables via `` `@databricks/sql` ``. The system communicates outward to Azure Communication Services for critical transactional email delivery (capped under strict outbound quotas), queries the Anthropic Claude API for synthetic

portfolio narrative summaries, and exposes dedicated HTTPS endpoints for push-based agent heartbeat and usage telemetry.

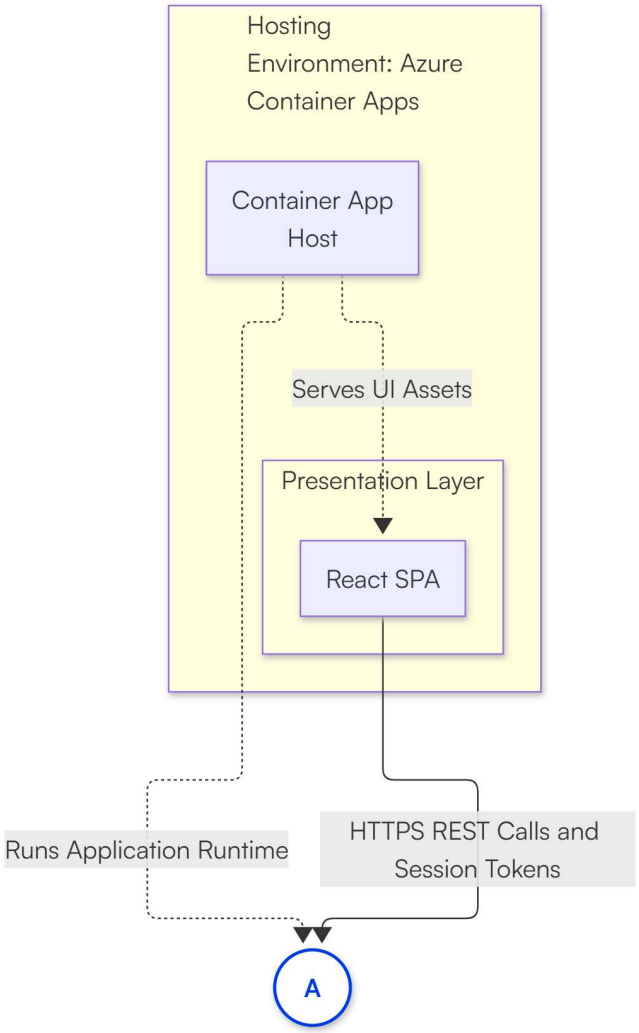


Figure 2 — Architecture (part 1 of 5)

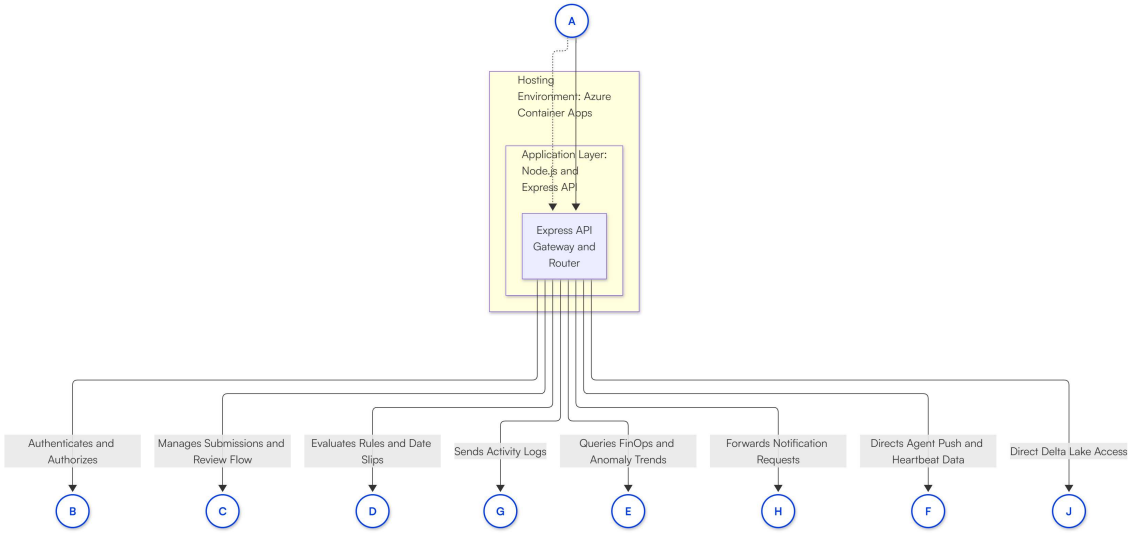


Figure 2 — Architecture (part 2 of 5)

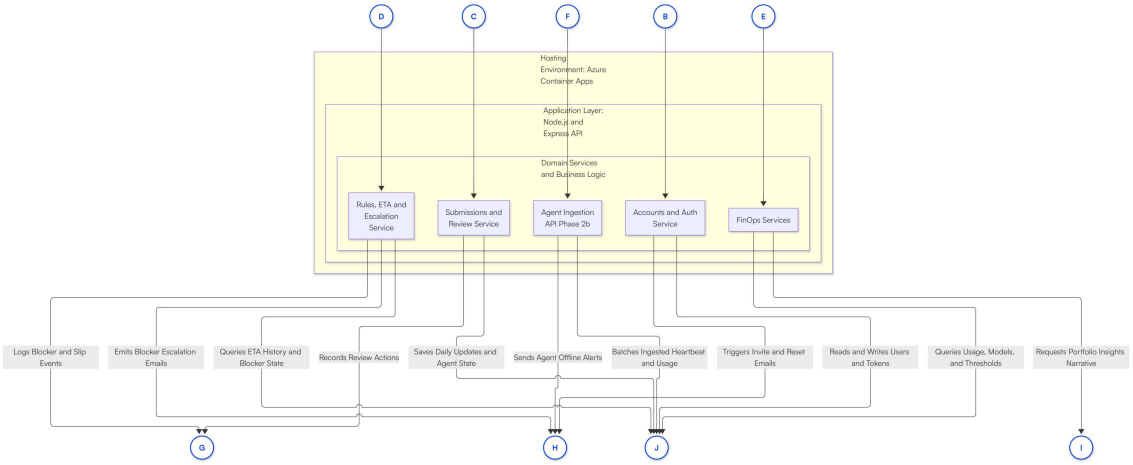


Figure 2 — Architecture (part 3 of 5)

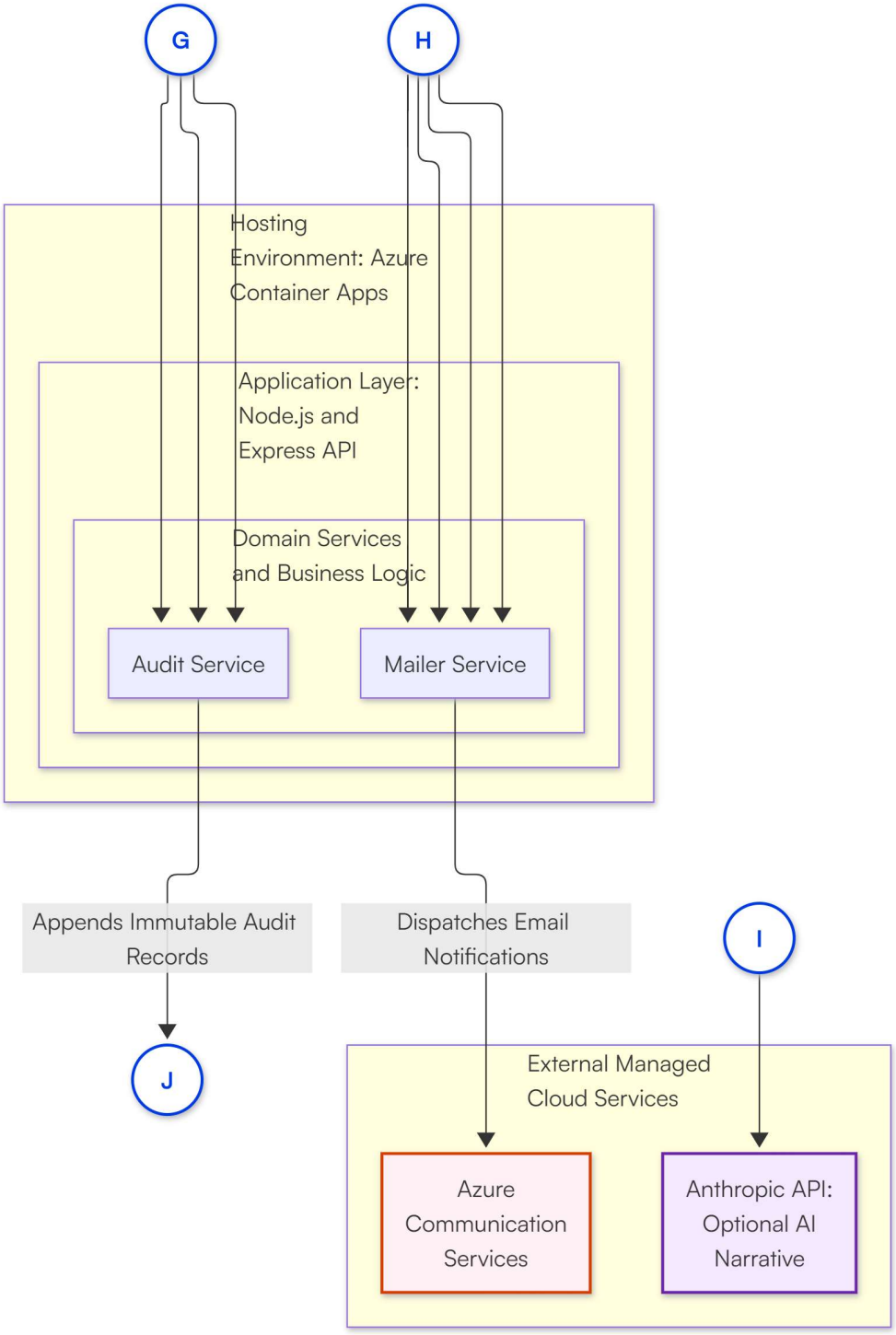


Figure 2 — Architecture (part 4 of 5)

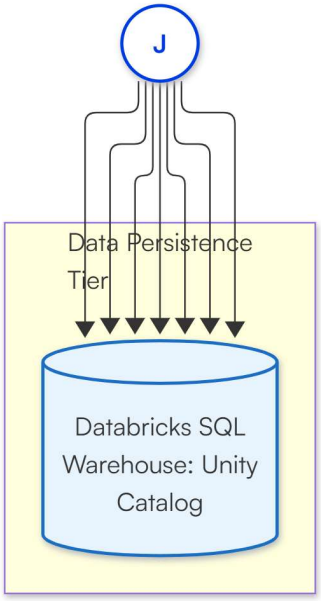


Figure 2 — Architecture (part 5 of 5)

5.3 Proposed Technology Stack

Layer	Proposal	Notes
Frontend UI	React 19, Vite, MUI, TanStack React Query, React Router, Recharts, react-hook-form	Vite provides optimized container image builds; MUI delivers enterprise UI components with native dark/light theme support; TanStack Query simplifies client-side caching, polling, and invalidation for review workflows.
Backend API	Node.js 20 LTS, Express, Plain JavaScript (ES Modules), Zod	Plain JavaScript avoids TypeScript build-step drift across monorepo packages; Node.js 20 LTS provides stable enterprise runtime execution; Zod guarantees strict schema validation at the API edge.

intuitive

Layer	Proposal	Notes
Persistence Store	Databricks SQL (Unity Catalog, Delta Lake) via `` `@databricks/sql` ``	Leverages pre-existing enterprise analytical infrastructure; eliminates the need to provision separate relational SQL instances without IT intervention; Delta Lake supports durable historical analytical tables.
Container Platform	Docker, Azure Container Apps, Azure Container Registry, Azure Log Analytics	Enables micro-container management without full Kubernetes orchestration overhead; co-hosts client SPA and Express server on a single origin to eliminate cross-origin complexity.
Outbound Messaging	Azure Communication Services (ACS)	Native Azure resource accessible via service connection strings; allows outbound email delivery on Azure-managed domains without provisioning corporate Exchange mailboxes or altering corporate DNS.
CI/CD & Automation	GitHub Actions, oxlint, npm workspaces	Monorepo workspaces manage frontend and backend boundaries cleanly; oxlint delivers sub-second linting; GitHub Actions builds release containers without holding live production database secrets.
Analytical AI Engine	Anthropic SDK (`` `@anthropic-ai/sdk` ``)	Optional integration to generate synthetic natural-language portfolio executive summaries; design includes deterministic algorithmic fallback when API tokens are omitted.

intuitive

5.4 Design Principles

- **Zero-IT-Dependency Footprint:** The architecture commits to zero operational demands on corporate IT; it requires no Entra ID application registrations, no DNS modifications, no dedicated service mailboxes, and no cloud infrastructure beyond pre-approved AI Garage subscription boundaries.
- **Strict Push-Only Ingestion Boundary:** The dashboard never initiates outbound connections to, nor scans, agents' proprietary hosting environments or underlying databases. Agents must push telemetry outward over TLS via standard HTTPS calls.
- **Metadata Exclusivity:** The system persists only operational, schedule, cost, and reliability metadata. No runtime business prompts, conversational completions, proprietary customer payloads, or agent training sets are ever ingested or stored.
- **Server-Authoritative Enforcement:** Security, authorization, and tenant isolation exist entirely on the server. Client-side routing guards serve solely as user-experience conveniences; API routes validate object ownership, assignment scopes, and role permissions on every request.
- **Non-Blocking Observability:** Failure of external analytical dependencies (e.g., Anthropic API limits, email delivery quotas, or Databricks query timeouts) must degrade gracefully and never prevent the recording of core agent updates or telemetry ingestion.

---

6. Non-Functional Requirements

6.1 Security

- User passwords must be salted and hashed using `scrypt` with a minimum requirement of 12 characters.
- Five consecutive failed sign-in attempts must lock the target account for exactly 15 minutes.
- User deactivation, administrative role modifications, or password reset completions must immediately revoke all active session tokens.
- Agent telemetry endpoints must authenticate requests via unique, per-agent API keys that can be revoked independently.
- Inbound API routes must enforce strict rate limiting and validate all payload structures using Zod schemas.

- No credentials, connection strings, or system secrets shall be committed to the code repository; all configuration is injected via Azure Container App runtime secrets.

6.2 Access Control

- Role-based permissions (Contributor, Agent Manager, Executive, FinOps) must be validated and enforced strictly at the API layer.
- Contributors requesting details or submitting updates for an unassigned agent must receive an HTTP 404 response to prevent agent enumeration.

6.3 Auditability

- An append-only audit log must record every structural, schedule, and metadata modification to an agent, capturing before-and-after values, timestamp, and actor identity.
- Audit records must preserve historical agent names and contributor identity strings even after the corresponding agent or user profile has been deleted from the registry.

6.4 Availability

- The Azure Container App deployment must maintain a minimum replica count of 1 and a maximum replica count of 1 once agent heartbeats are active, preventing multi-instance polling anomalies and ensuring offline detection remains deterministic.

6.5 Performance

- Portfolio dashboard views and summary tables must achieve complete rendering within 3 seconds under normal network conditions.
- Agent telemetry and usage ingestion endpoints must batch writes to Databricks SQL Delta tables to prevent query queue exhaustion and handle burst traffic.

6.6 Data Integrity

- The database layer must enforce application-level unique constraints (such as one recorded daily update per agent per calendar day) via Databricks Delta `MERGE` statements.
- Partial-failure behavior across multi-table updates must be documented, with transactional state self-healing on the subsequent submission cycle.

intuitive

6.7 Usability

- The client user interface must be fully responsive across viewport widths down to mobile form factors.
- The SPA must support both light and dark display modes.
- Every user-initiated state change must yield immediate visual toast feedback confirming success or describing server errors.

6.8 Testability

- Unit testing suites, static linting (`oxlint`), and frontend production builds must pass in GitHub Actions CI prior to merging code.
- Full integration test suites must execute successfully against the isolated `agent-dashboard-test` schema prior to promoting any release to production.

6.9 Cost

- Operational runtime costs must remain minimal, leveraging serverless Azure Container App tiers and consumption-priced Azure Communication Services (~$0.00025 per sent email).

6.10 Maintainability

- Backend code must follow a strict layered architectural pattern (Route -> Controller -> Service -> DB access) implemented in clean, documented ES-native JavaScript.

---

7. Risks

ID	Risk	Impact	Mitigation
R-01	Databricks connection resets (ECONNRESET) and slow integration runs (~25 minutes)	Medium	Document restart guidance; implement reconnect retry handling; require the suite to run before each deploy.

intuitive

ID	Risk	Impact	Mitigation
R-02	Non-atomic writes can leave an agent's current values stale	Medium	Document the tradeoff; design subsequent contributor updates to overwrite and correct inconsistent state.
R-03	Single reviewer is a bottleneck	Medium	Reviewer is modeled as a functional permission rather than a hardcoded role; delegate review capabilities to team leads later.
R-04	Email limits or deliverability on the Azure-managed domain	Medium	Provide copy-to-clipboard fallback links for user invites; rate-limit alert generation to stay within quota.
R-05	Migration 013 backfills blocked-since, so long-blocked live agents are escalated on the first hourly check	Medium	Warn the Agent Manager prior to running production deployment so notifications can be anticipated.
R-06	Azure tag policies break deployments	Medium	Update the required `CreatedMonth` and `Practice` tags programmatically in deployment scripts prior to resource provisioning.
R-07	Low adoption by contributors	Medium	Keep the contributor experience focused on the streamlined "My Agents" interface; schedule automated reminder notifications in Phase 3.

intuitive

ID	Risk	Impact	Mitigation
R-08	Agent teams do not integrate with 2b	Medium	Provide copy-paste multi-language telemetry snippets; make onboarding documentation and visibility prominent.
R-09	Token or key leakage	Medium	Protect tokens via Azure Container App secret references; utilize isolated per-agent keys that can be instantly revoked.
R-10	No SSO until IT approves	Medium	Accept secure, isolated local account authentication with scrypt hashing and enforced lockout until IT can be engaged.

8. Delivery Phases

Phase	Content	Exit Criteria
Phase 1	Agent tracking, daily updates, FinOps, Azure deployment	Live
Phase 2a	Accounts, submit and review, My Agents, audit log, ETA slips, blocker escalation, tests and CI	Built and tested on the test schema; plan finalised 2026-10-13; live deploy paused
Phase 2a deploy	Migrations 010—013, email setup, rollout, merge to main	[target date]
Phase 2b	Per-agent API keys, HTTPS push, heartbeat, offline alerts, integration snippets	Planned [dates]

intuitive

Phase	Content	Exit Criteria
Phase 3	Weekly executive summary, Teams notifications, A2A test tasks, control features, Microsoft sign-in	Future

Execution dependencies are strictly linear: Phase 2a deployment must complete on the production schema before Phase 2b telemetry services can be rolled out, and Phase 3 depends on the accumulated historical usage, reliability, and status data generated across Phases 2a and 2b.

---

9. Open Questions

- Who becomes a reviewer besides the Agent Manager, and when?
- Which team owns onboarding agents to the 2b push integration?
- What baselines and targets should the success metrics use?
- Should the weekly executive summary come before 2b?
- When is an agent considered 'production' and so expected to send heartbeats?
- What is the retention policy for audit, usage and heartbeat data?
- Will IT approve an Entra app registration for Microsoft sign-in in Phase 3?

---

10. Future Scope (Deferred, Not Rejected)

Nothing in this section is committed, estimated, or approved for current development sprints.

intuitive

Item	What it would unlock	What it would require
Weekly executive summary email	Automated distribution of portfolio progress, active blockers, and weekly spend summaries directly to stakeholders.	Phase 2a and 2b production baseline data; scheduling worker/cron; verification of ACS volume limits.
Microsoft Teams notifications	Real-time broadcast of blocker escalations, review approvals, and offline alerts into team channels.	Microsoft 365 tenant webhook access or Bot Framework app permissions (requires IT involvement).
A2A test tasks	Automated inter-agent workflow testing to validate integration capabilities across disparate AI workloads.	Completed Phase 2 visibility; standardized agent RPC/REST interfaces; orchestration test harness.
Control features (start/stop agents)	Ability for operators to remediate failing or runaway agents directly from the dashboard interface.	High-trust governance baseline; secure remote execution endpoints on agents' native hosting infrastructure.
Microsoft sign-in (SSO)	Seamless corporate single sign-on; eliminates individual passwords and simplifies onboarding/offboarding.	Corporate IT sponsorship; Azure Entra ID app registration approval; tenant client secret provisioning.
Multi-role users / role switcher	Flexibility for users who act both as departmental contributors and high-level portfolio reviewers.	Redesigned authorization context; UI persona-switching mechanism; updated API session claims.

intuitive

Item	What it would unlock	What it would require
CI running integration tests	Automated verification of Databricks database operations on every GitHub pull request.	Storing Databricks warehouse credentials/secrets within GitHub Actions environments (policy decision).

---

11. Current Deployment Status

Phase 1 is live on Azure Container Apps. The Phase 2a deploy is paused partway through.

Completed Items:

- Azure Communication Services resources and sender domain provisioning
- ACS email connection string configuration as an application secret
- Azure resource compliance tagging (Practice tags applied)

Remaining Items:

- Execution of database migrations 010 through 013 against the live Databricks schema
- Container image compilation, tag versioning, and rollout to Azure Container Apps
- Configuration of production environment variables (`APP_URL`, `EMAIL_SENDER`, `ACS_CONNECTION_STRING`)
- Deprecation and removal of the legacy shared-password environment variable
- Generation of the initial administrative account invite link
- Merging release branches to `main` in source control
- Re-executing the complete integration test suite against the `agent-dashboard-test` schema prior to resuming deployment

intuitive

12. Sign-Off

Role	Name	Signature	Date
Executive Sponsor	[Name]	_____	_____
Agent Manager	Arya Kayastha	_____	_____
FinOps Representative	[Name]	_____	_____